ESE 5370: Final Project Report

Parth Sarkar

Monday, April 20th, 2026

1 Introduction

Memory safety is a key priority for programs written in C. The burden of programing defensively against malicious use of undefined behavior lies squarely on the programmer, leading to mistakes that can be exploited by user input to the program. While the problem is pervasive and fundamentally a consequence of C semantics, the Softbound+CETS project offers the strong, formally-verified guarantee of *full* memory safety. Softbound+CETS instruments C code to create and update metadata for each pointer in the program at runtime. Upon a dereference for a pointer, the pointer's metadata is cross-referenced to ensure that the dereference occurs "within bounds" and is not a use-after-free.

However, this approach has two limitations. First, from a performance perspective, it is highly expensive to branch and check any condition on every dereference; [INSERT (revisited)] cites a 140% overhead on the SPEC CPU 2017 C benchmark suite. However, more pressingly, Softbound+CETS does not offer binary-level compatibility. Hence, if any C code that has not been compiled with Softbound+CETS instrumentation is linked into a protected project, the entire executable is susceptible to memory privilege escalation.

With these problems in mind, my final project first attempts to concretize the problem of privilege escalation due to foreign-linked code with a working attack that permits a buffer-overflow write (one that would normally be stopped by Softbound+CETS instrumentation if compiled with protection). As a proposed solution, I implement and compare two encryption methods on the metadata table that Softbound+CETS uses to propagate metadata on pointers stored into memory: a simple XOR encryption, and a more complicated AES encryption method. On the performance front, I implement and formally prove important properties of a global dataflow analysis in LLVM 12 and the Rocq interactive theorem prover, respectively. I integrate the results of this analysis into the Softbound+CETS pass and demonstrate correctness and tangible performance improvements. Ultimately, I hope my project makes the case that dynamic enforcement with strong security guarantees can go hand-in-hand with provably correct, classical static analysis methods, so the security-performance tradeoff can continue to be bridged.

2 The Attack

In order to motivate the encryption of the metadata table that is central to Softbound+CETS' memory safety guarantees, my project includes a simple attack embedded in an executable linked with one compilation unit unprotected by Softbound+CETS instrumentation. At a high level, given

knowledge about the location in virtual memory of the metadata table, the unprotected module can directly modify select entries of the table in order to escalate the privilege of a specific pointer. In my case, I have demonstrated this fact by modifying the table to allow, within the protected module, an offset from a stack allocated `Buffer A` to modify a disjoint section of the stack owned by `Buffer B`. While this is a simple example, I think it's clear that this is a limitation to the adoption of SoftBound+CETS instrumentation in legacy systems. Why should "protected" modules incur considerable overheads when there is no guarantee that data cannot be corrupted by pointers that live in and never even leave the protected module?

2.1 Relevant Background

First, in order to understand the way the attack was mounted, it is important to specify the mechamism that SoftBound+CETS uses to propagate and update metadata associated with each pointer.

2.1.1 Pointer & Metadata Origination

C semantics dictates that pointers can originate in multiple, legitimate ways. Most straightforwardly, the compiler can allocate variable, fixed-size buffers, and other statically-sized objects on the function's stack frame. Pointer variables that point to stack-allocated memory have a base, bound, and size that is determined at compile time. This metadata is inserted in-line with the pointer variable declaration.

In particular, here is my mental model of stack variable pointer transformations. With the following code:

```
struct student {
  int age;
  int id;
};

int main()
{
  struct student parth;
  parth.age = 23;
}
```

I expect to see LLVM intermediate representation code to the following effect:

```
...
%1 = alloca %struct.student, align 4
%1_age = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 0

// Softbound+CETS inserted
%1_base = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 0
%1_bound = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 2
%1_size = i32 4; // each field of 'student' holds 4 bytes
...
```

Here, the `getelementptr` (GEP) instruction is just computing a pointer offset from some base. Then, at the point where the variable `%1_age` is finally dereferenced, a branch is also inserted to verify the condition `%1_base <= %1_age + %1_size <= %1_bound`. In this example (where `fp` stands for "frame pointer"), the check simplifies to:

```
fp + 0 <= fp + 0 + 4 <= fp + 8
```

2.1.2 Pointer Propagation

An interesting case to deal with is when programmers need to store pointer variables themselves into memory (as a concrete example, the `task_struct` in the Linux kernel holds a pointer to the `pid` struct in its field `thread_pid`).

Here, SoftBound+CETS instrumentation is required to make a *metadata table entry* for the pointer value being stored to memory. The metadata cannot be reliably inlined because the pointer value will most likely escape the scope that any inlined metadata might belong to.

In particular, the metadata table maps each pointer stored into memory to a struct with `base` and `bound` values. The mapping uses a two-level table in order to balance quick look-up and modification times with reasonable memory usage. Specifically, the container I used to mount the attack used 48 bit pointers. The following diagram shows the way a pointer value is used to find its mapped metadata entry:

2.2 Attack Details

My attack employs an unprotected module to overwrite a key metadata table entry to escalate the privilege afforded to a pointer that lives in the SoftBound+CETS instrumented subset of the executable. This is not the only way to modify a pointer's metadata. For example, even the "inlined" metadata presented in 2.1.1 will exist in the memory if `main` calls a subroutine. A malicious subroutine might write beyond its stack frame, and modify the stack location pertaining to the `%1_bound` variable in its parent's stack frame.

However, my attack exploits the metadata table because I wanted to motivate the encryption of the metadata table as an initial, low-complexity encryption target (and get a rough understanding of overhead). It is possible (and necessary, for full protection) to encrypt metadata that lives on the stack, but that was a more involved process that will be left to future work.

In my attack, I initialize two adjacent arrays, `array1` and `array2`, and I store the pointer to `array1`'s first element is stored to memory.

```
/* main.c (protected) */

// these arrays live next to each other on the stack
int array1[] = {1, 2, 3, 4};
int array2[] = {5, 6, 7, 8};

// ensure pointer 'array1' has an entry in the metadata table by
// storing the pointer to a location in memory (i.e. not just inlined)
int *array1_ptr[] = {array1};
```

Next, in my unprotected module, having turned off ASLR, I hard-coded the address of the first-level metadata table. I then descended the two-level table, found the entry, and widened my privilege by 4 bytes in either direction.

In my protected module, I then used my escalated privilege to write beyond the bounds of `array1`, and influence `array2` (the '37' seen in the image below). The directory with the code can be accessed here: `https://github.com/parthsarkar17/softboundcets-ese5370/tree/main/attack`.